

Kasino-korttipeli dokumentti

Yleistiedot

Nimi: Aapo Tynninen

Opiskelijanumero: 100674912

Koulutusohjelma: Sähkötekniikan kandidaattiohjelma 2. vsk
(Elektroniikka ja sähkötekniikka)

Päiväys: 8.5.2025

Yleiskuvaus

Projektin ideana oli toteuttaa Pythonilla kasino-korttipeli graafisella käyttöliittymällä. Suunnitelmasta poiketen projekti toteutettiin keskivaikealla vaikeustasolla.

Kasinon säännöt

Tavoite:

- Pelissä kerätään pisteitä, jotka lasketaan aina jokaisen pelikierroksen lopussa.
- Peli jatkuu, kunnes joku pelaajista saavuttaa 16 pistettä.

Pelikierros:

- Jokaisen pelikierroksen alussa pakka sekoitetaan ja jakaja jakaa jokaiselle pelaajalle 4 korttia (eivät näy muille) sekä 4 korttia pöytään (näkyvät kaikille). Loput kortit jätetään pöydälle pinoon ylösalaisin.
- Jakajasta seuraava aloittaa pelaamisen. Seuraavalla pelikierroksella hän on jakaja.

Mitä kukin pelaaja tekee vuorollaan:

- Pelaaja voi kerrallaan käyttää jonkin kädessään olevista korteista: joko ottaa sillä pöydästä kortteja tai laittaa kortin pöytään. Jos pelaaja ei voi ottaa mitään pöydästä täytyy hänen laittaa jokin korteistaan pöytään.
- Jos pelaaja ottaa pöydästä kortteja hän kerää ne itselleen pinoon. Pinon sisällöstä lasketaan korttien loputtua pisteet.

- Pöydässä olevien korttien määrä voi vaihdella vapaasti. Jos joku vaikkapa ottaa kaikki kortit, täytyy seuraavan laittaa jokin korteistaan tyhjään pöytään.
- Aina käytettyään kortin, pelaaja ottaa käteensä pakasta uuden, niin että kädessä on aina 4 korttia. (Kun pöydällä oleva pakka loppuu, ei oteta enää lisää vaan pelataan niin kauan kuin kenelläkään on kortteja kädessä. Tällöin siis tietenkin alle 4 korttia on mahdollinen tilanne.)
- Kortilla voi ottaa pöydästä yhden tai useampia samanarvoisia kortteja ja kortteja, joiden summa on yhtä suuri, kuin kortin jolla otetaan. Seuraava esimerkki selventää asiaa:

Mökki:

- Jos joku saa kerralla pöydästä kaikki kortit kerralla, hän saa ns. mökin, joka merkitään muistiin.

Erikoiskortit:

- Pelissä on muutama kortti, joiden arvo kädessä on arvokkaampi kuin pöydässä:
 - Ässät: kädessä 14, pöydässä 1
 - Ruutu-10: kädessä 16, pöydässä 10
 - Pata-2: kädessä 15, pöydässä 2
- Esim. ruutukympillä voi siis ottaa pöydästä vaikkapa kaksi kahdeksikkoa.
- Muiden korttien arvot ovat samat sekä pöydässä että kädessä.

Pistelasku:

Kun kaikilta loppuvat kädestä kortit saa viimeksi pöydästä kortteja ottanut loput pöydässä olevat kortit. Tämän jälkeen lasketaan pisteet ja lisätään ne entisiin pisteisiin.

Seuraavista asioista saa pisteitä:

- Jokaisesta mökistä saa yhden pisteen
- Jokaisesta ässästä saa yhden pisteen
- Eniten kortteja saanut saa yhden pisteen
- Eniten patoja saanut saa 2 pistettä
- Ruutu-10-kortin omistaja saa 2 pistettä
- Pata-2-kortin omistaja saa pisteen

Käyttöohje

Ohjelma käynnistetään ajamalla Koodit-kansiosta tiedosto [main.py](#). Ohjelma ei vaadi erillisiä komentoriviargumentteja tai asetustiedostoja. Ohjelman käynnistyessä aukeaa aloitusikkuna, josta voi aloittaa uuden pelin tai jatkaa vanhaa tallennettua peliä, mikäli sellainen on olemassa. Ohjelmasta voi poistua millä tahansa hetkellä painamalla ESC-näppäintä. Ohjelma avaa vahvistusikkunan, jossa voi vahvistaa tai perua poistumisen.

Uuden pelin aloittamisen jälkeen avautuu pelaajienlisäysikkuna, jossa voi syöttää 2-N pelaajan nimeä ohjelmalle. Kun vähintään kaksi pelaajaa on lisätty, pelin voi aloittaa.

Pelin alettua aukeaa peli-ikkuna, jossa näkyy vuorossa olevan pelaajan kortit sekä pöydässä olevat kortit. Ikkunan vasemmassa laidassa on tapahtumat-palsta, johon ilmestyy pelin tapahtumia sitä mukaa kun kortteja nostetaan pöydästä tai laitetaan pöytään. Vuorossa oleva pelaaja valitsee omista korteistaan yhden pelattavan painamalla kortin kuvaketta. Jos pelaaja haluaa nostaa pöydästä kortteja, hän painaa haluamiensa korttien kuvakkeita. Jos hän haluaa laittaa kortin pöytään, hän jättää pöydän kortit valitsematta. Kun pelaaja on valinnut haluamansa kortit, hän painaa valmis-nappia. Tällöin ohjelma tarkistaa korttien laillisuuden. Jos kortit ovat lailliset, ohjelma nostaa pelaajalle automaattisesti pakasta uuden kortin, jos niitä on jäljellä. Tämän jälkeen vuoro siirtyy seuraavalle pelaajalle viiden sekunnin kuluessa. Peli on mahdollista lopettaa millä tahansa hetkellä painamalla ESC-näppäintä tai "pala päävalikkoon" -näppäintä. Tällöin ohjelma avaa ikkunan, jossa voi päättää pelin tallennuksesta ja perua lopettamisen.

Aloitusikkunassa painettaessa "jatka peliä" -nappia ohjelma siirtyy suoraan peli-ikkunaan, jossa on kaikki äsken mainitut toiminnallisuudet.

Yksi kierros jatkuu niin pitkään, kunnes jokaisen pelaajan käsi on tyhjä. Tämän jälkeen aukeaa kierroksenpäättösikkuna, josta näkee pelaajien tilastot kierrokselta. Kierroksenpäättösikkunassa on mahdollisuus lopettaa peli tai jatkaa. Jos päättää lopettaa pelin, ohjelma avaa ikkunan, jossa voi päättää pelin tallentamisesta ja perua lopettamisen. Jos päättää jatkaa, ohjelma avaa uuden peli-ikkunan ja pelaajien tilastot nollataan lukuunottamatta pisteitä.

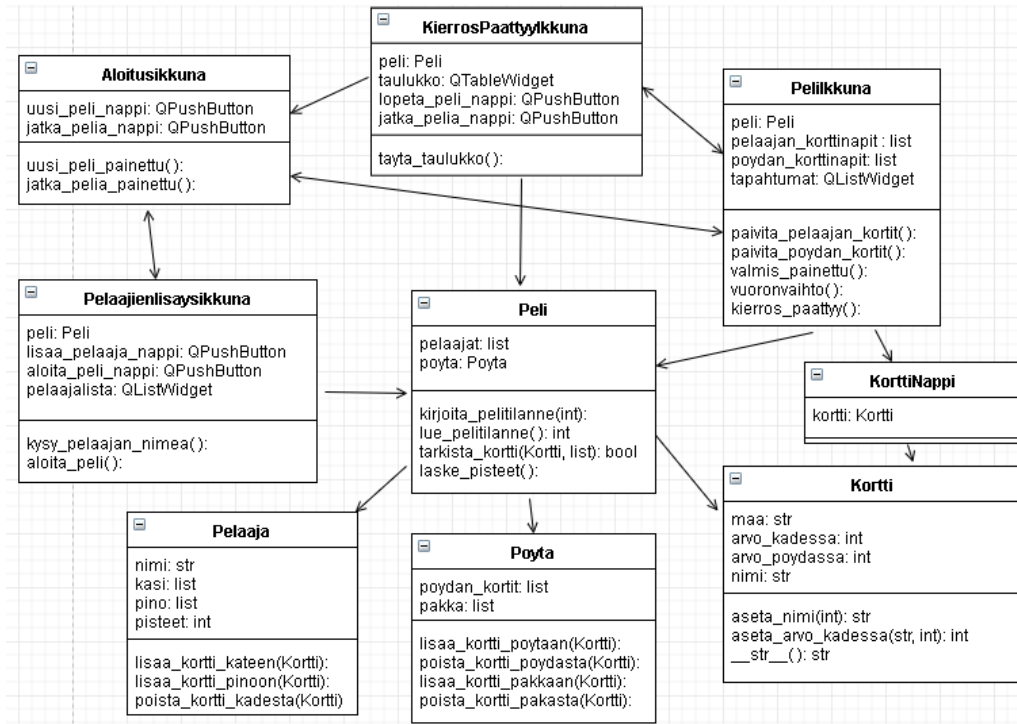
Peliä voi jatkaa siihen asti, että joku pelaajista saa kerättyä 16 pistettä. Tällöin kierroksenpäättösikkuna ei enää tarjoa mahdollisuutta jatkaa tai tallentaa peliä. Pelistä voi poistua päävalikkoon tai sulkea ohjelman kokonaan.

Ulkoiset kirjastot

Projektissa on käytetty ainoastaan PyQt6-kirjastoa sekä Pythonin mukana tulevaa random-moduulia.

Ohjelman rakenne

Lopullinen UML-luokkakaavio:



Ohjelman yksinkertaisimmat perusluokat ovat Pelaaja, Poyta ja Kortti.

Pelaaja-luokka varastoi pelaajan nimen, kortit ja pisteet, Poyta-luokka kaikki pöydällä olevat kortit ja Kortti-luokka kaiken yksittäisen kortin sisältämän tiedon. Luokkien metodit ovat yksinkertaisia ja aika itsestäänselviä.

Poyta-luokan olisi periaatteessa voinut sisällyttää Peli-luokkaan, mutta koska Peli on jo valmiiksi ohjelman suurin luokka, niin uskon tämän olevan parempi ratkaisu. Harkitsin myös korttien toteuttamista sanakirjana, mutta koska yksi kortti sisältää paljon informaatiota, kuten maa, nimi ja kaksi eri arvoa, niin päädyin toteuttamaan sen luokkana.

Ohjelman selkäranka on Peli-luokka, joka tuo edellä mainitut perusluokat yhteen ja vastaa suurimmasta osasta ruudun takana tapahtuvaa toimintaa. Pelin tallennukseen ja lataamiseen tarkoitetut metodit "kirjoita_pelitilanne" ja "lue_pelitilanne" käsittelevät tekstitiedostoja. Kirjoitusmetodi kirjoittaa jokaisen pelaajan tiedot omille riveilleen ja lopuksi pöydällä olevat kortit. Lukumetodi lukee tiedot samassa järjestyksessä ja alustaa tietojen mukaiset pelaajat, kortit, pöydän ja pelin. "Tarkista_kortti" tarkistaa nimensä mukaisesti korttien laillisuuden, eli vastaako

pelaajan pelaama kortti hänen pöydästä valitsemiaan kortteja sääntöjen puitteissa. "Laske_pisteet" käy läpi jokaisen pelaajan korttipinon ja antaa pelaajille pisteitä sääntöjen mukaisesti.

Graafinen käyttöliittymä on toteutettu viidellä luokalla: AloitusIkkuna, Pelaajienlisäysikkuna ja PeliIkkuna, jotka perivät luokan QMainWindow, KierrosPaattyyIkkuna, joka perii luokan QWidget sekä KorttiNappi, joka perii luokan QPushButton. QMainWindow-luokan perivät luokat sisältävät kaikki keyPressedEvent-metodin, jolla mahdollistetaan pelistä poistuminen ESC-näppäimellä.

Aloitusikkuna sisältää napit "uusi peli" ja "jatka peliä". Nappien toiminnallisuus hoidetaan metodeilla "uusi_peli_painettu", joka avaa pelaajienlisäysikkunan ja "jatka_peliä_painettu", joka luo Peli-olion, lukee pelitilanteen tekstitiedostosta ja avaa peli-ikkunan, mikäli tallennettu peli on olemassa. Muuten se avaa virheviesti-ikkunan.

Pelaajienlisäysikkuna sisältää napit "lisää pelaaja", "aloita peli" ja "takaisin päävalikkoon". Näitä varten on määritetty metodit "kysy_pelaajan_nimeä", "aloita_peli" ja "palaa". "Kysy_pelaajan_nimeä" avaa tekstikentän, johon kirjoitetaan lisättävän pelaajan nimi. Lisätyn pelaajan nimi lisätään listaan. Mikäli annettu nimi on 0 tai yli 25 merkkiä pitkä tai annettu nimi on jo pelaajalistassa, avautuu virheviesti-ikkuna. "Aloita_peli" avaa peli-ikkunan tai virheviesti-ikkunan, mikäli pelaajia on lisätty alle 2. "Palaa" avaa ensin vahvistusikkunan ja sen jälkeen uuden aloitusikkunan.

PeliIkkunassa on napit jokaiselle pelaajan omalle kortille sekä pöydän kortille. Lisäksi on "valmis"-nappi, "palaa päävalikkoon"-nappi ja "tapahtumat"-lista. Luokka sisältää metodit pelaajan ja pöydän korttinappien päivittämiseksi sekä yksittäisten nappien lisäämiselle ja poistamiselle. "Valmis"-napin painaminen kutsuu "valmis_painettu"-metodia, joka tarkistaa kortit ja korteista riippuen poistaa tai lisää kortteja pöytään sekä nostaa pelaajalle uuden kortin, jos niitä on pakassa vielä jäljellä. "Vuoronvaihto"-metodi tarkistaa, että pitääkö kierrosta vielä jatkaa ja päivittää viiden sekunnin viiveellä seuraavan vuorossa olevan pelaajan kortit näytölle. Mikäli kierros tulee päättyä, "kierros_paattyy"-metodi lisää pöytään jääneet kortit viimeisimmälle nostajalle ja avaa KierrosPaattyyIkkunan.

KierrosPaattyyIkkuna sisältää taulukon sekä napit "lopetta peli" ja "jatka peliä". "Tayta_taulukko"-metodi nimensä mukaisesti käy läpi pelaajien tiedot ja täyttää ne taulukkoon. Metodi myös tarkistaa, että onko kellään pelaajista yli 16 pistettä ja jos on, niin peli päätetään.

KorttiNappi sisältää ainoastaan kortti-ilmentymämuuttujan. Tämä siksi, että silmukalla luodut napit voitaisiin tunnistaa listasta.

Algoritmit

Ohjelman tärkein algoritmi on korttientarkistusalgoritmi, jota käytetään Peli-luokan "tarkista_kortti"-metodissa. Parametreina saadaan pelaajan pelaama kortti sekä hänen pöydästä valitsemansa kortit. Algoritmi käy erilaiset tapaukset läpi järjestyksessä yksinkertaisesta monimutkaiseen. Jos pöydästä valittuja kortteja on yksi, tarkistetaan suoraan onko se samanarvoinen kuin pelattu kortti. Jos kortteja on useampi, tarkistetaan ensin onko niiden summa samanarvoinen kuin pelattu kortti. Jos ei, tarkistetaan onko joukossa pelatun kortin kanssa samanarvoisia kortteja ja poistetaan ne joukosta. Tämän jälkeen tarkistetaan onko jäljelle jääneiden korttien summan arvo pelatun kortin arvon monikerta. Jos on, tarkistetaan kaikki mahdolliset jäljellä olevien korttien osajoukkojen summat. Mikäli summa on samanarvoinen kuin pelattu kortti, osajoukon jäsenet lisätään uuteen joukkoon. Kun kaikki osajoukot on käyty läpi, verrataan jäljellä olevia kortteja ennen läpikäyntiä ja uutta joukkoa.

Pseudokoodiesitys algoritmille:

```
jos pöydästä valittuja kortteja on 1
  jos pelattu kortti = valittu kortti
    tosi
  muuten
    epätosi
jos pöydästä valittujen korttien summa = pelattu kortti
  tosi
jos mikään valituista korteista = pelattu kortti
  poista kortti valituista korteista
jos uusi pöydästä valittujen korttien summa = pelattu kortti
  tosi
jos uusi pöydästä valittujen korttien summa on pelatun kortin monikerta
  käydään osajoukot läpi
    jos osajoukon summa = pelattu kortti
      lisää osajoukon jäsenet uuteen joukkoon
  jos uusi pöydästä valitut kortit = uusi joukko
    tosi
  muuten
    epätosi
```

Tietorakenteet

Suurimmaksi osaksi ohjelmassa käytetään tietorakenteena listoja, jotka ovat muuttuvarakenteisia. Listat sopivat ohjelman käyttöön hyvin, koska monissa ohjelman toiminnoissa vaaditaan muuttuvarakenteista tietorakennetta, joka soveltuu hyvin tietyn tietotyypin alkioden varastointiin. Myös indeksointi on tarpeellinen ominaisuus useissa ohjelman osissa. Toisaalta useimmissa listoissa ei toistu sama alkio, joten joukkokin voisi myös toimia, jos indeksointia ei tarvita. Joukkoa hyödynnettiin korttientarkistusalgoritmin lopussa, koska korttien vertailuun tarvittiin tietorakennetta, jossa on vain uniikkeja alkioita ja jossa järjestyksellä ei ole väliä.

Parissa metodissa on käytetty arvojen palautukseen myös monikkoa. Monikko on kätevä useamman kuin yhden arvon palauttamiseen, jos määrä on kiinteä, koska sitä ei tarvitse erikseen luoda ja lisätä siihen alkioita kuten esimerkiksi listassa.

Tiedostot

Ohjelma käsittelee ainoastaan tekstitiedostoja, joihin pelitilanne on tallennettu. Tiedosto seuraa pitkälti hyvin yksinkertaista formaattia: ensin tulee yksi-kaksimerkkinen kokonaisluku, joka ilmaisee datan pituuden ja seuraavana itse data. Ensimmäisenä tulee yksimerkkinen luku, joka ilmaisee seuraavan luvun pituuden. Tämä luku on ns. indeksi, joka pitää kirjaa pelaajien vuoroista pelissä. Indeksien jälkeen tulee toinen yksimerkkinen luku, joka ilmaisee aloitusvuoron pituuden, jonka jälkeen tulee aloitusvuoro. Sen jälkeen jokaisen pelaajan tiedot on kirjoitettu omalle rivilleen tyylillä kaksimerkkinen datan pituus -> data ja viimeisellä rivillä on pöydän kortit ja pakan kortit. Pelaajan tiedot on kirjoitettu järjestyksessä: nimi, mökit, pisteet, käsi, pino. Viimeisellä rivillä tulee ensin pakka ja sitten pöydän kortit. Kortit on kirjoitettu formaatilla {maa}-{nimi}. Numerokorteille nimi on vain arvo merkkijonomuodossa ja kuvakorteille "Jatka", "Kuningatar", "Kuningas" ja "Assa". Aina yhden kortteja sisältävän datalohkon jälkeen tulee vakio "DATALOHKO_PAATTYY", joka on arvoltaan 99. Kun viimeisen pelaajan pinolohko loppuu, vakio tulee kaksi kertaa peräkkäin, jonka jälkeen alkaa pakan korttien lohko. Vakio voi myös luonnollisesti esiintyä kaksi kertaa peräkkäin, jos joku kortteja sisältävistä lohkoista on tyhjä.

Testaus

Ohjelman testaus hoidettiin muutamalla yksikkötestillä sekä ohjelmaa ajamalla. Eniten yksikkötestejä tein korttientarkistusalgoritmille, koska se on hyvin tärkeä osa ohjelmaa ja suhteellisen yksinkertainen testata. Tein yksikkötestit neljälle erilaiselle tapaukselle, jotka kattavat lähes jokaisen metodin rivin. Ensimmäisessä pöydästä valittuja kortteja on vain yksi, toisessa kaksi korttia joiden summa on yhtä suuri kuin pelattu kortti, kolmannessa kolme korttia joista yksi on yhtä suuri kuin pelattu kortti ja myös kahden muun summa on yhtä suuri ja neljännessä viisi korttia, joista yksi on yhtä suuri kuin pelattu kortti ja neljästä muusta tietyt kahden kortin summat ovat yhtä suuria.

Tein kaksi yksikkötestiä pelitilanteen luvulle ja -kirjoitukselle. Tein testit siinä vaiheessa, kun käyttöliittymää ei ollut vielä aloitettu, joten testit eivät olleet kovin kattavia, koska peliä ei voinut vielä varsinaisesti pelata. Kuitenkin näillä testeillä saatiin lähes kaikki virheet korjattua ja loput korjattiin ohjelman normaalin käytön yhteydessä.

Tein myös hyvin yksinkertaiset testit kortin pöytäänlaittometodille ja pisteenlaskumetodille. Tarkoituksena oli lähinnä saada selville, että voiko ohjelma kaatua näiden metodien ansiosta.

Suuren osan testaamisesta tein yksinkertaisesti peliä pelaamalla. Ohjelmassa oli ennen käyttöliittymän toteutusta muutamia kohtia, joita ei ollut testattu lainkaan,

koska peliä ei voinut vielä pelata. Kun käyttöliittymä alkoi valmistua, virheitä oli jonkin verran, mutta kaikki saatiin korjattua.

Yritin kirjoittaa myös yksikkötestejä käyttöliittymän osille, mutta ohjelman kaatuessa terminaaliin ei tulostunut tracebackia, vaan pelkkä exit-koodi. Tämä hankaloitti käyttöliittymän testausta huomattavasti. Vasta myöhään selvisi, että kun ohjelman ajaa komentoriviltä, niin se pakottaa tracebackin tulostuksen. Tämän avulla sain onneksi viimeisetkin virheet korjattua.

Projektisuunnitelmaan kirjoittamani testaussuunnitelma ei ollut kovin kattava, jonka vuoksi lopullista testausta oli jonkin verran enemmän.

Ohjelman puutteet ja viat

Ohjelman käyttöliittymässä peli-ikkunassa pöydän korttien asettelu ei ole kovinkaan optimaalinen. Kortteja lisätään ensin yhteen riviin, kunnes kun kortteja on yli kahdeksan, jolloin siirrytään seuraavaan riviin. Jos alemmalla rivillä on kortteja, niin vaikka ylemmästä rivistä otettaisiin kortteja, alemman rivin kortit pysyvät alemmalla rivillä. Myös korttien määrä ylärivillä lasketaan korttien kokonaismäärästä, joten jos alemmalla rivillä on kortteja, niin ylemmälle riville ei voi enää lisätä yhtä montaa korttia.

Käytin paljon aikaa siihen, että olisin saanut asettelun toimimaan oikein käyttäen QGridLayoutia, mutta kortit menivät aina välillä päällekkäin enkä saanut vikaa millään korjattua. Siksi päädyin tähän huonompaan, mutta toimivaan ratkaisuun. Ja toisaalta kasinossa on kuitenkin loppujen lopuksi aika harvoin yli kahdeksan korttia pöydässä.

Lisäksi pelaajienlisäysikkunasta puuttuu mahdollisuus poistaa lisättyjä pelaajia. Ominaisuus olisi ollut hyödyllinen esimerkiksi tilanteissa, jossa käyttäjä kirjoittaa vahingossa pelaajan nimen väärin. Sen toteuttaminen olisi kuitenkin vaatinut aika paljon erilaisilla napeilla ja asetteluilla kikkailemista, joten päätin jättää sen tekemättä. Pelaajienlisäysikkunasta voi kuitenkin poistua päävalikkoon ja avata uuden pelin, joka korvaa ominaisuuden edes jossain määrin.

Paras ja heikoin kohta

Mielestäni ohjelman paras kohta on korttientarkistusalgoritmi. Suunnitteluvaiheessa kuvittelin algoritmin vievän hirveän määrän rivejä, mutta siitä tuli loppujen lopuksi aika kompakti. Lisäksi mielestäni osajoukkojen läpikäyminen käyttäen bittitason operaatioita oli aika nerokas idea.

Heikoin kohta ohjelmassa on edellä mainittu pöydän korttien asettelu.

Poikkeamat suunnitelmasta

Toteutin projektin vaativan vaikeustason sijasta keskivaikealla vaikeustasolla, koska projektin keskivaiheilla en pystynyt käyttämään tarpeeksi aikaa projektin edistämiseen. Erityisesti maaliskuun loppu ja huhtikuun alkupuoli olivat minulle odotettua kiireisempiä. Siksi projektin loppuun ei jäänyt tarpeeksi aikaa tehdä tietokonevastustajaa.

Tästä syystä en myöskään kerennyt panostamaan graafisen käyttöliittymän ulkonäköön niin paljon kuin olisin halunnut. Loppujen lopuksi siitä tuli ihan hyvä, mutta se olisi voinut olla parempikin. Käyttöliittymän luokkajako poikkesi aika paljonkin suunnitelmasta, mutta se johtuu yksinomaan siitä, että minulla ei ollut suunnitteluvaiheessa vielä mitään kokemusta PyQt:sta.

Peli-luokka ei myöskään ollut suunnitelmassa. Kuitenkin perusluokkien toteuttamisen jälkeen huomasin, että ohjelma tarvitsee välttämättä luokan, joka tuo perusluokat yhteen ja pyörittää peliä ruudun takana.

.csv-tiedoston sijaan päädyin käyttämään pelitilanteen tallennuksessa tekstitiedostoa, koska en ollut suunnitteluvaiheessa tajunnut miettiä sitä, että jotta koko pelitilanne saadaan tallennettua, niin jokainen kortti pitää kirjoittaa tiedostoon erikseen. Tämän hoksattuani päädyin valitsemaan tekstitiedoston paremmaksi vaihtoehdoksi tehtävään.

Suunniteltu toteutusjärjestys osui oikeaan, mutta ajankäyttöarvio ei niinkään. Perusluokat valmistuivat nopeasti ja korttientarkistusalgoritmikin vei yllättävän vähän aikaa. Kuitenkin käyttöliittymän aloitus tapahtui monta viikkoa myöhemmin kuin piti ja se olikin loppujen lopuksi projektin eniten aikaa vievä osuus.

Toteutunut työjärjestys ja aikataulu

Perusluokat Kortti, Pelaaja ja Pöytä olivat pitkälti valmiita jo ensimmäisen välipalautuksen kohdalla 13.3. Kolmanteen välipalautukseen 20.3. mennessä korttientarkistusalgoritmi oli valmis ja Peli-luokkakin ylipäättään jo hyvällä mallilla. Tiedostoonkirjotus- ja tiedostotalukumetodit valmistuivat kuudennella välipalautuksella 10.4. Sen jälkeen olin viikon matkoilla. 24.4. mennessä main-funktio ja graafinen käyttöliittymä oli saatu aloitettua. Tässä poikettiin useammalla viikolla suunnitelmasta, sillä graafinen käyttöliittymä olisi pitänyt aloittaa jo kuudennella palautusviikolla. 2.5. käyttöliittymä oli jo hyvällä mallilla, mutta jonkin verran tekemistä vielä jäljellä. Peli-luokka ja graafinen käyttöliittymä kaikkine lisäyksineen ja korjauksineen tulivat valmiiksi vasta viimeisellä viikolla.

Yhteenveto

Ohjelman käyttöliittymä on hyvin selkeä ja helppokäyttöinen. Ohjelmaa pystyisi varmasti käyttämään helposti ilman erillisiä käyttöohjeita, edellyttäen että tuntee kasinon säännöt. Virhetilanteet käsitellään siten, että käyttäjä varmasti ymmärtää mikä meni vikaan. Huonoina puolina käyttöliittymässä on kuitenkin pari edellämainittua puutetta korttien asettelun ja pelaajienlisäysikkunan kanssa. Mitään hyvin olennaisia puutteita ohjelmassa ei kuitenkaan ole.

Ohjelmaa voisi parantaa ensinnäkin korjaamalla edellä mainitut huonot puolet. Käyttöliittymän estetiikkaa voisi myös parantaa. Lisäksi ohjelmaan voisi lisätä tietokonevastustajan ja esimerkiksi mahdollisuuden tallentaa useampia pelejä. Uskon, että ohjelman rakenne on hyvin soveltuva näiden parannusten tekemiseen. Esimerkiksi tietokonevastustajan tapauksessa voisi tehdä luokan, joka perii Pelaaja-luokan. Tietokonevastustaja käyttäisi pelin kulkiessa kaikkia samoja metodeita ja funktioita mitä normaalit pelaajat, mutta sille pitäisi vain koodata oma korttiervalinta-algoritmi.

Viitteet:

- <https://www.geeksforgeeks.org/iterating-over-all-possible-combinations-in-an-array-using-bits>
- <https://www.youtube.com/watch?v=HQNisfb795A>
- <https://docs.python.org/3/library/random.html>

Liitteet

Aloitusikkuna:



Pelaajienlisäysikkuna:

Lisää pelaaja

Aloita peli

Takaisin päävalikkoon

Lisätyt pelaajat:

Peli-ikkuna:

Palaa päävalikkoon

Tapahtumat

Pöydän kortit

Omat kortit (Pelaaja "pelaaja 1")

Valmis